

Modelado Técnico del Sistema para Plataforma E-commerce de Periféricos de PC

1. Introducción

El presente documento técnico detalla el modelado del sistema para la plataforma de comercio electrónico de periféricos de PC, utilizando diagramas de componentes y de clases. El objetivo es proporcionar una visión clara de la estructura interna y las interacciones entre los microservicios, facilitando la comprensión del diseño técnico y sirviendo como guía para el desarrollo y la implementación. Se analizará la estructura de alto nivel del sistema y se profundizará en la organización interna de dos microservicios críticos: el Servicio de Catálogo y el Servicio de Pedidos.

Este modelado es fundamental para asegurar la coherencia en el desarrollo, la mantenibilidad del código y la correcta integración de los diferentes módulos, en línea con los principios de una arquitectura de microservicios.

2. Diagrama de Componentes del Sistema

El diagrama de componentes ofrece una vista de alto nivel de la arquitectura del sistema, mostrando los principales bloques funcionales (microservicios) y sus interacciones. Este diagrama ilustra cómo los diferentes servicios colaboran para ofrecer la funcionalidad completa de la plataforma.

Modelado Técnico del Sistema para Plataforma E-commerce de Periféricos de PC

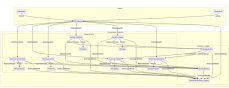
1. Introducción

El presente documento técnico describe el modelado del sistema para la plataforma de comercio electrónico de periféricos de PC, ofreciendo diagramas de componentes y de clases. El objetivo es proporcionar una visión clara de la estructura interna y las interacciones entre los microservicios, facilitando la comprensión del desarrollo y el mantenimiento del sistema y su evolución a lo largo del tiempo.

Este modelado no solo define la arquitectura, sino que también establece las responsabilidades de cada componente, asegurando la coherencia y el cumplimiento de los requisitos del negocio y de los usuarios.

2. Diagrama de Componentes del Sistema

El diagrama de componentes ofrece una vista de alto nivel de la arquitectura del sistema, mostrando los principales bloques funcionales (microservicios) y sus interacciones. Este diagrama ilustra cómo los diferentes servicios colaboran para ofrecer la funcionalidad completa de la plataforma.



Descripción del Diagrama de Componentes:

- Clientes (Web y Móvil):** Interactúan con la plataforma a través de la API Gateway.
- API Gateway:** Punto de entrada unificado, responsable del enrutamiento, autenticación/autorización y gestión de la carga.
- Microservicios:** Componentes autónomos que encapsulan una funcionalidad de negocio específica (Autenticación, Carrito, Pedidos, Pagos, Notificaciones).
- Servicio de Mensajería:** Permite la comunicación asíncrona entre microservicios y el consumo de eventos.
- Message Broker:** Facilita la comunicación asíncrona entre microservicios mediante eventos, promoviendo el desacoplamiento.
- Base de Datos:** Cada microservicio posee su propia base de datos, optimizada para sus necesidades específicas de almacenamiento.
- Sistema de Monitoreo y Logging:** Recopila métricas y logs de todos los componentes para observabilidad y diagnóstico.

3. Modelado Interno del Microservicio de Catálogo

El Servicio de Catálogo es responsable de gestionar la información de los productos, categorías y sus disponibilidad. Su diseño interno se basa en una arquitectura limpia, separando las preocupaciones en capas bien definidas.

Interacción con el Sistema:

- API Gateway:** Exponer endpoints REST para la consulta de productos y categorías.
- Message Broker:** Consumir eventos de otros servicios (ej. "Stock Actualizado") desde el Servicio de Pedidos para mantener la consistencia del inventario.
- Base de Datos:** Utiliza una base de datos NoSQL (ej. MongoDB) para almacenar productos y categorías, aprovechando la flexibilidad de esquema para atributos de productos variables.

Diagrama de Clases del Servicio de Catálogo:



Descripción de Clases Clave:

- ProductInterface:** Define la interfaz para las operaciones CRUD de productos y categorías.
- ProductRepository:** Implementa la lógica de acceso a la base de datos (MongoDB) para productos.
- ProductService:** Contiene la lógica de negocio para productos, incluyendo validaciones y lógica de negocio.
- ProductEntity:** Representa la estructura de datos de un producto.
- CategoryEntity:** Representa la estructura de datos de una categoría.
- CategoryService:** Gestiona la lógica de negocio para categorías.

4. Modelado Interno del Microservicio de Pedidos

El Servicio de Pedidos es responsable de la creación, gestión y seguimiento del ciclo de vida de los pedidos. Su diseño interno sigue una estructura en capas para una clara separación de responsabilidades.

Interacción con el Sistema:

- API Gateway:** Exponer endpoints REST para la creación de pedidos, consulta de historial y seguimiento de estado.
- Message Broker:** Publicar eventos (ej. "Pedido Creado", "Pedido Actualizado") para notificar a otros servicios (ej. Servicio de Inventario, Servicio de Notificaciones). Consumir eventos (ej. "Pago Exitoso") del Servicio de Pagos.
- Servicio de Catálogo:** Realizar llamadas sincronicas para obtener detalles de productos al crear un pedido.
- Base de Datos:** Utiliza una base de datos relacional (ej. PostgreSQL) para asegurar la integridad transaccional de los pedidos y sus items.

Diagrama de Clases del Servicio de Pedidos:



Descripción de Clases Clave:

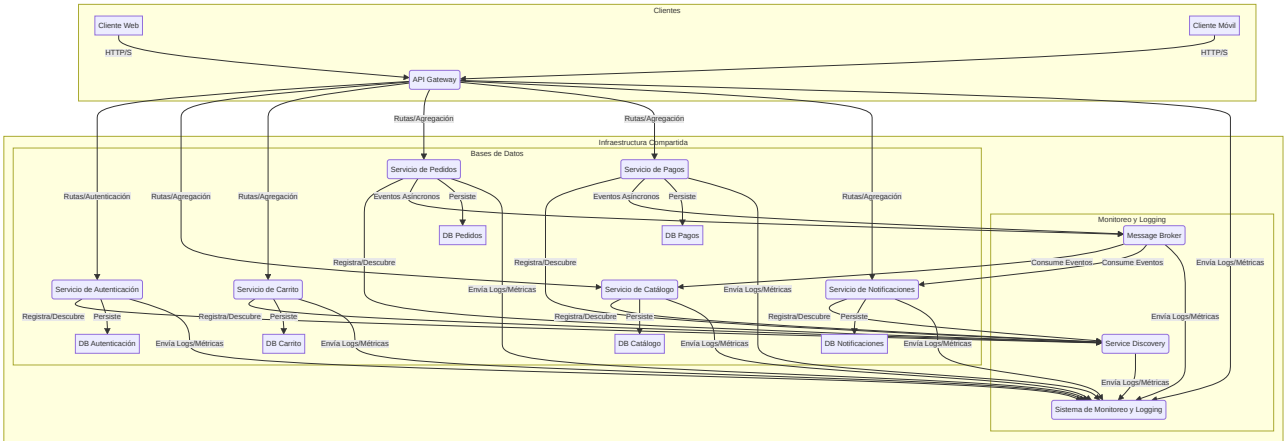
- OrderInterface:** Define la interfaz para las operaciones CRUD de pedidos.
- OrderRepository:** Implementa la lógica de acceso a la base de datos (PostgreSQL) para pedidos.
- OrderService:** Contiene la lógica de negocio para pedidos, incluyendo validaciones y lógica de negocio.
- OrderEntity:** Representa la estructura de datos de un pedido.
- ItemEntity:** Representa la estructura de datos de un ítem de un pedido.
- PaymentEntity:** Representa la estructura de datos de un pago.

5. Conclusión

El modelado técnico presentado en este documento, a través de diagramas de componentes y de clases, proporciona una comprensión profunda de la estructura y el funcionamiento de la plataforma de comercio electrónico de periféricos de PC. La clara definición de las responsabilidades de cada microservicio y la organización interna de sus clases principales facilitan el desarrollo, el mantenimiento y la escalabilidad del sistema. Este documenta es un herramienta esencial para el equipo de desarrollo, asegurando que la implementación se alinee con el diseño arquitectónico y los requisitos de negocio.

6. Referencias

- Field, M. (2020). *101 Questions About the Standard Object Model*. Lulu.com.
- Reisner, J. (2020). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Evans, E. (2018). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Microservices Patterns.



Descripción del Diagrama de Componentes:

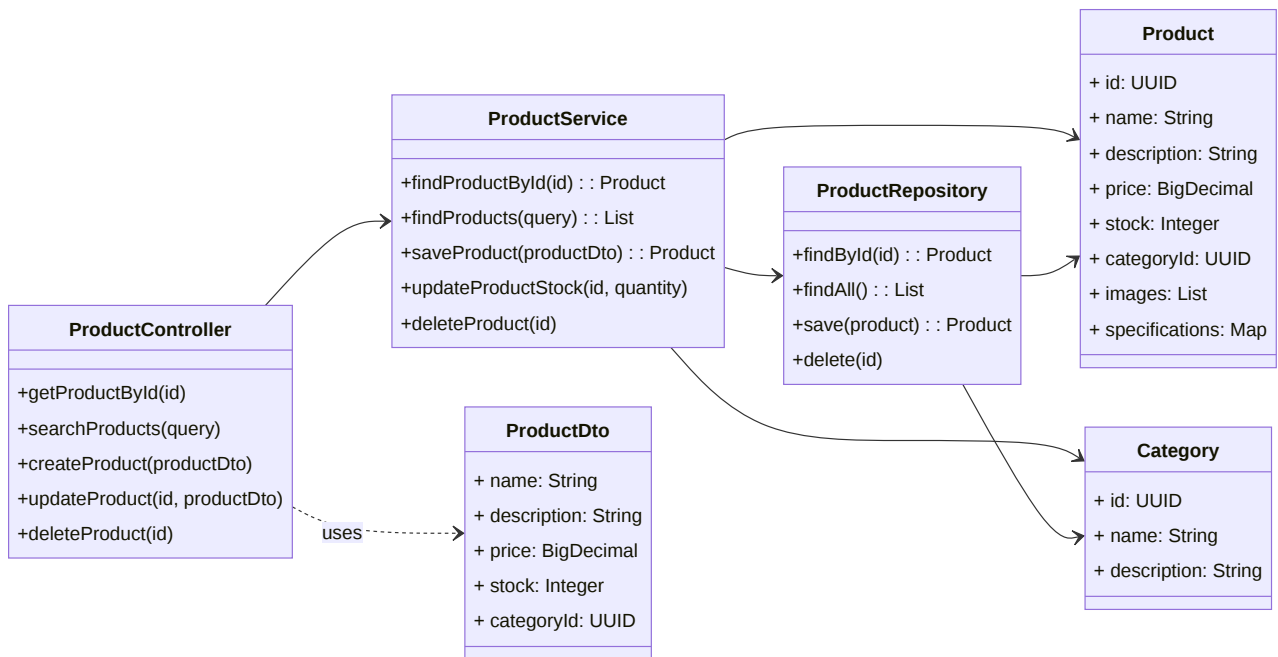
- **Clientes (Web y Móvil):** Interactúan con la plataforma a través del API Gateway.
- **API Gateway:** Punto de entrada unificado, responsable del enrutamiento, autenticación/autorización y agregación de respuestas.
- **Microservicios:** Componentes autónomos que encapsulan una funcionalidad de negocio específica (Autenticación, Catálogo, Carrito, Pedidos, Pagos, Notificaciones).
- **Service Discovery:** Permite a los microservicios registrarse y encontrarse dinámicamente.
- **Message Broker:** Facilita la comunicación asíncrona entre microservicios mediante eventos, promoviendo el desacoplamiento.
- **Bases de Datos:** Cada microservicio posee su propia base de datos, optimizada para sus necesidades (persistencia polígloa).
- **Sistema de Monitoreo y Logging:** Recopila métricas y logs de todos los componentes para observabilidad y diagnóstico.

3. Modelado Interno del Microservicio de Catálogo

El Servicio de Catálogo es responsable de gestionar la información de los productos, categorías y su disponibilidad. Su diseño interno se basa en una arquitectura limpia, separando las preocupaciones en capas bien definidas.

Integración con el Sistema:

- **API Gateway:** Expone endpoints RESTful para la consulta de productos y categorías.
- **Message Broker:** Consume eventos de otros servicios (ej. "Stock Actualizado" desde el Servicio de Pedidos) para mantener la consistencia del inventario.
- **Base de Datos:** Utiliza una base de datos NoSQL (ej. MongoDB) para almacenar productos y categorías, aprovechando su flexibilidad de esquema para atributos de producto variables##### Diagrama de Clases del Servicio de Catálogo:



Descripción de Clases Clave:

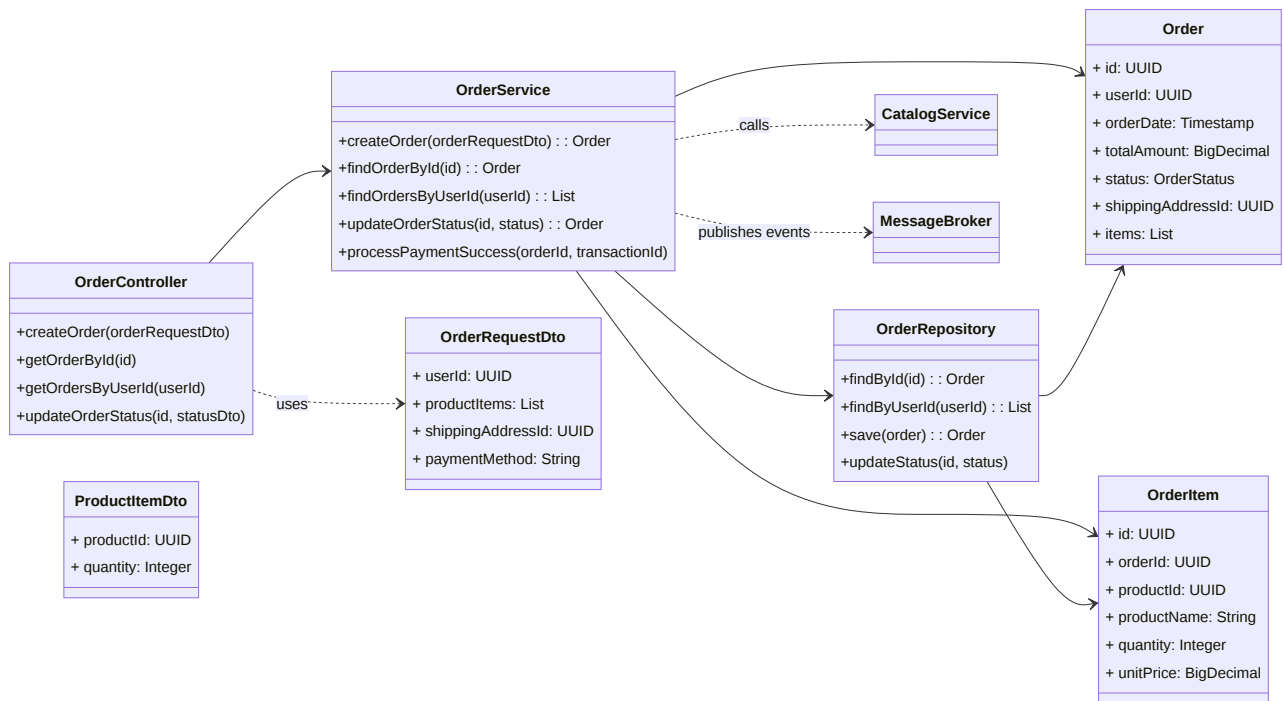
- **ProductController** : Expone la API REST para las operaciones CRUD de productos y la búsqueda. Actúa como la interfaz de entrada del microservicio.
- **ProductService** : Contiene la lógica de negocio principal. Coordina las operaciones de repositorio, aplica reglas de negocio (ej. validación de stock) y puede interactuar con otros servicios (ej. publicar eventos de stock).
- **ProductRepository** : Abstrae la lógica de acceso a la base de datos (MongoDB). Realiza las operaciones de persistencia para las entidades **Product** y **Category**.
- **Product** : Entidad de dominio que representa un periférico, incluyendo sus atributos como nombre, precio, stock, especificaciones, etc.
- **Category** : Entidad de dominio que representa una categoría de productos.
- **ProductDto** : Objeto de transferencia de datos utilizado para la entrada y salida de la API, desacoplando la representación externa de la interna.

4. Modelado Interno del Microservicio de Pedidos

El Servicio de Pedidos es responsable de la creación, gestión y seguimiento del ciclo de vida de los pedidos. Su diseño interno también sigue una estructura en capas para una clara separación de responsabilidades.

Integración con el Sistema:

- **API Gateway:** Expone endpoints RESTful para la creación de pedidos, consulta de historial y seguimiento de estado.
- **Message Broker:** Publica eventos (ej. "Pedido Creado", "Pedido Actualizado") para notificar a otros servicios (ej. Servicio de Inventario, Servicio de Notificaciones). Consume eventos (ej. "Pago Exitoso" del Servicio de Pagos).
- **Servicio de Catálogo:** Realiza llamadas síncronas para obtener detalles de productos al crear un pedido.
- **Base de Datos:** Utiliza una base de datos relacional (ej. PostgreSQL) para asegurar la integridad transaccional de los pedidos y sus ítems##### Diagrama de Clases del Servicio de Pedidos:



Descripción de Clases Clave:

- **OrderController** : Expone la API REST para la gestión de pedidos. Recibe solicitudes del API Gateway.
- **OrderService** : Contiene la lógica de negocio para la creación, actualización y consulta de pedidos. Interactúa con **OrderRepository** , realiza llamadas al Servicio de Catálogo para obtener detalles de productos y publica eventos a través del **MessageBroker** .
- **OrderRepository** : Abstrae la lógica de acceso a la base de datos (PostgreSQL). Gestiona la persistencia de las entidades **Order** y **OrderItem** .
- **Order** : Entidad de dominio que representa un pedido, incluyendo su estado, fecha, monto total y una lista de ítems de pedido.
- **OrderItem** : Entidad de dominio que representa un producto específico dentro de un pedido, con su cantidad y precio unitario en el momento de la compra.
- **OrderRequestDto** y **ProductItemDto** : Objetos de transferencia de datos para la creación de pedidos, desacoplando la representación externa de la interna.

5. Conclusión

El modelado técnico presentado en este documento, a través de diagramas de componentes y de clases, proporciona una comprensión profunda de la estructura y el funcionamiento de la plataforma de e-commerce de periféricos de PC. La clara definición de las responsabilidades de cada microservicio y la organización interna de sus clases principales facilitan el desarrollo modular, la mantenibilidad y la escalabilidad del sistema. Este documento es una herramienta esencial para el equipo de desarrollo, asegurando que la implementación se alinee con el diseño arquitectónico y los requisitos de negocio.

6. Referencias

- [Fowler, M. \(2003\). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* \(3rd ed.\). Addison-Wesley.](#)
- [Newman, S. \(2015\). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.](#)
- [Evans, E. \(2003\). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.](#)
- [Mermaid Documentation.](#)